

Operating System

Assignment-4

Short Answer type: Part-A

1. A race condition happens when two people try to use a shared resource at the same time, causing inconsistent results.

Example: Two people withdrawing money from the same bank account at the same time - both see ₹ 5000, both withdraw ₹ 5000, balance becomes wrong.

Mutual exclusion: Allows only one person / process to access the resource at a time, preventing conflicting actions.

- **Peterson's Solution:** Pure software; simple conceptually but works reliably only on single-core systems. Needs strict memory ordering → hardware dependent in practice.

- **Semaphores:** Supported directly by OS / hardware atomic instructions, easier to implement in real systems, works on multi-core machines.

Monitors automatically bundle data + synchronization, so locking is handled implicitly. This avoids programmer errors and reduces context-switch overhead, making concurrency safer in multi-core

4. Starvation: If writers keep arriving, readers may be blocked forever (or vice versa).

Prevention: Use fair scheduling, e.g. a queue where requests are served in order of arrival (FIFO).

5. Processes must request all resources upfront, causing:

- low resource utilization (many resources stay idle).
- larger waiting times $\rightarrow$ poor system performance.

Part B : Application / Numerical Based

6. Given:

S1 : $P_1 \rightarrow P_2$, $P_3 \rightarrow P_4$

S2 : $P_2 \rightarrow P_5$, $P_5 \rightarrow P_6$

S3 : $P_6 \rightarrow P_1$

(a) Global graph:

$P_1 \rightarrow P_2 \rightarrow P_5 \rightarrow P_6 \rightarrow P_1$ (cycle)

and $P_3 \rightarrow P_4$ (no cycle)

(b) Deadlock? Yes

Deadlocked processes: P_1, P_2, P_5, P_6

(c) Algorithm:

Chandy - Misra - Haas algorithm for distributed deadlock detection.

7. Local = 5ms

Remote = 25ms

Probability remote = 0.3

Probability local = 0.7

(a) Expected time:

$$0.7 \times 5 + 0.3 \times 25$$

$$= 11 \text{ ms}$$

(b) Caching strategy:
client-side caching with LRU $\rightarrow$ frequently accessed files stay local, reducing remote access and improving speed.

Full = 200ms

Incremental = 50ms

RPO = 1 second $\rightarrow$ must checkpoint at least every 1 sec.

(a) optimal plan (for 10 seconds):

Do 1 full checkpoint at the start + incremental checkpoint every 1 second.

Total: 1 full + 9 incremental

(b) Reasoning:

Full provides a clean base; incrementals every 1 sec keep overhead low ($50\text{ms} \ll 200\text{ms}$) and meet RPO.

9. (a) Challenges:

sudden traffic spikes, uneven load across regions, varying network latency.

Suitable algorithm:

Dynamic load balancing with least loaded or Work-stealing.

(b) Use multi-region replication + automatic failover (e.g., active-active data centres)

- RTO: very low because another region takes over instantly.
- RPO: near zero using continuous replication.